BTW GLOBAL DISTRIBUTORS
YOUR GLOBAL SUPPLY PARTNER

SUPPLY CHAIN, LOGISTICS, INVENTORY & SALES PLATFORM

Platform architecture & processes

What has been built so far: the system's structure, how a consignment moves from plan to reconciliation, how data flows, who uses what, and the data kept by each module. Version 1.1 · 18 October 2026 · source of truth: the code in this repository.

New in 1.1. Period plans are suggestions (Active / Replaced, no approval, no overlaps per company). Planners issue POs; director approval is a switch in `apps/api/config.toml`. Loading may differ from the PO within ±10% (or load alternatives), which creates a PO revision such as `…-rev001`. New **Truck planner** role. Office roles also get the **team phone app** (PWA) at `/app/`.

3
apps on one server: office web, phone app (field roles + office team PWA), API

11
roles, incl. truck planner; 3 field-only: loading point, driver, receiver

36
database tables (PostgreSQL + PostGIS)

6
integrations behind adapters: SMS/USSD, email, push, files, PDF, routing

Contents

1. Architecture diagram
2. Architecture explained
3. Process diagram (by actor)
4. Process & reconciliations explained
5. Data flow diagram
6. Use case diagram
7. Data structures per module (2 pages)

How to read the diagrams

Request / main flow — moving navy arrow (a call, a step done in order)

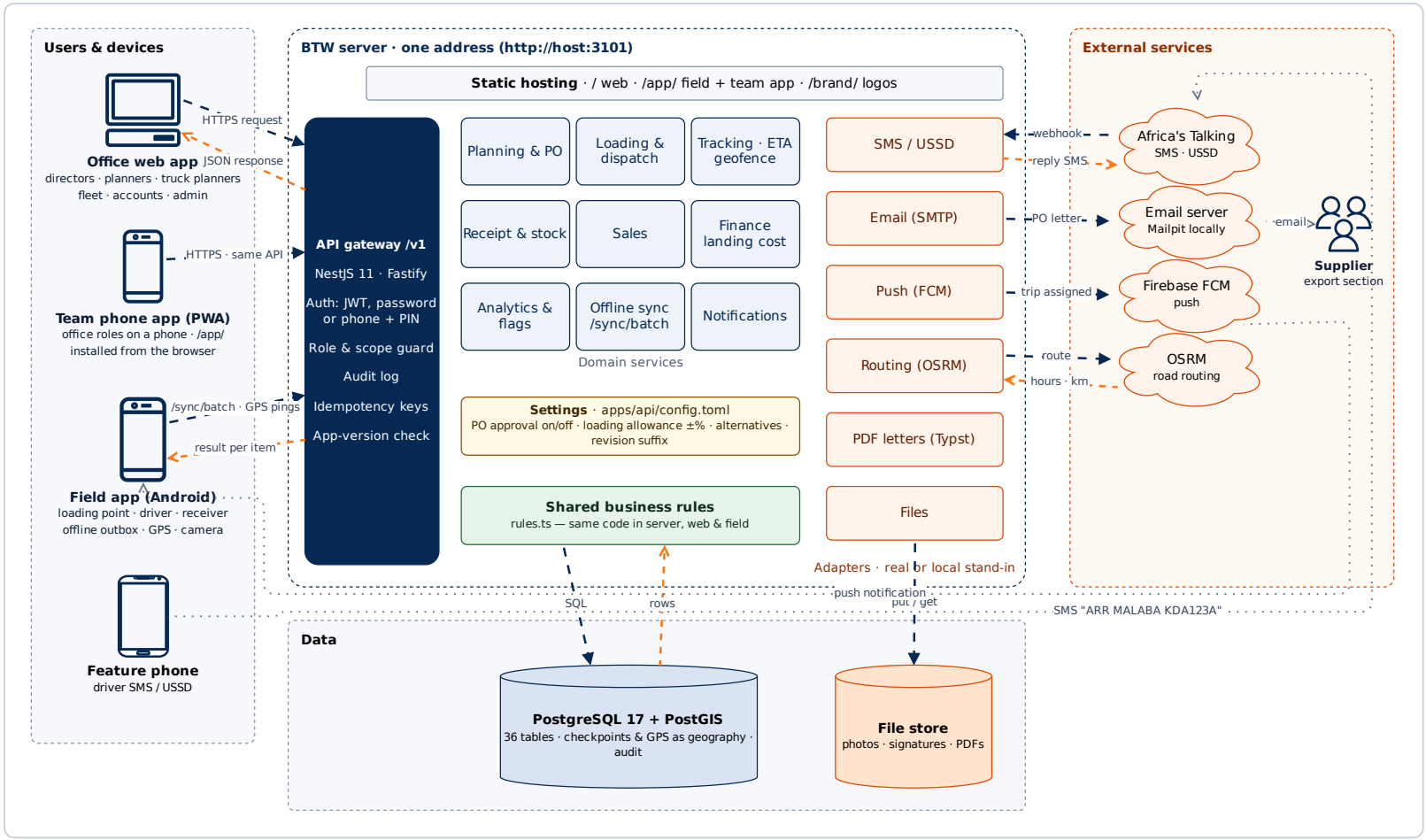
Response — moving dashed orange arrow (data returned)

Hand-over / notification — moving dotted arrow (between people, push, SMS, email)

The arrows move on screen (this HTML file, or the `.drawio` files opened in draw.io / diagrams.net, where each diagram can be edited). In print or PDF they appear as static dashed lines. Click a diagram to open it full screen.

One server, light apps, swappable integrations

diagrams/01-architecture.drawio



Clients. Office staff use the web app, or the team phone app (PWA) at `/app/`; the three field roles use the Android app, which queues work offline and sends it in one batch. Drivers without data can check in by SMS or USSD.

Server. One NestJS process serves the apps and the API at the same address. Every request passes auth, role and scope checks and is written to the audit log before a domain service runs the shared business rules. Business switches come from `config.toml`.

Data & integrations. PostgreSQL/PostGIS holds all operational data, including checkpoints and GPS. Each external service sits behind an adapter: without a key it uses a local stand-in, so nothing changes in code when keys are added.

1 · ARCHITECTURE

Architecture explained

Layers

LAYER	WHAT IT IS	TECHNOLOGY
Office web app	Period plans, POs and revisions, loading queues, Truck desk, shipments, tracking map, stock, sales, finance, analytics, master data, users & devices.	Svelte 5 + Vite · 112.7 KB first load · lazy map and charts
Field app	Map-first, one action at a time: *Hand over* (from the PO, ±allowance meter, alternatives) · *I've arrived / I'm leaving* · *Confirm receipt*. Offline outbox, location, camera, push.	Svelte 5 + Capacitor 7 (Android) · 73.5 KB
Team phone app (PWA)	Office roles at `/app/`: bottom tabs per role, side drawer, home with menu, KPIs and "Needs attention". Installable from the browser.	`apps/field/src/team`
API gateway	Sign-in (password or phone + PIN), rotating refresh tokens, remote device sign-out, role & scope guard (entity, loading point, warehouse), audit, idempotency keys, minimum app version.	NestJS 11 on Fastify · JWT · bcrypt
Domain services	Planning & PO, logistics & tracking, stock, sales, finance, analytics, notifications, offline sync. All share one rules file with the web and field apps.	TypeScript · `packages/api/src/rules.ts`
Data	33 tables; nested items (plan lines, checkpoint passes, issues, acknowledgement lines) stored as JSON inside their parent record; GPS and geofences as PostGIS geography.	PostgreSQL 17 + PostGIS 3.5 · Drizzle ORM
Integrations	SMS/USSD (Africa's Talking), email (SMTP), push (Firebase), files (disk / S3), PDF letters (Typst), routing (OSRM). Each has a real adapter and a local stand-in.	Adapters selected from `.env`
Business settings	PO approval on/off (default off), loading allowance ±10%, alternatives, revision suffix `-rev`. Sent to the apps in `GET /v1/ref`.	`apps/api/config.toml` (or `CONFIG_FILE`)

Life of a request — a driver's check-in

1. The driver slides *I've arrived at Malaba*; the app stores it in the outbox with a unique key and, when online, sends `POST /v1/sync/batch`.
2. The gateway checks the token, role and that the leg belongs to this driver, audits the call, and the idempotency check applies a retried item once only.
3. The tracking service records the pass (source: driver, geofence, SMS or office), recalculates leg status and ETA, and raises alerts.
4. A result per item comes back; the app clears the outbox. Office screens see the change on their next refresh.

Running it

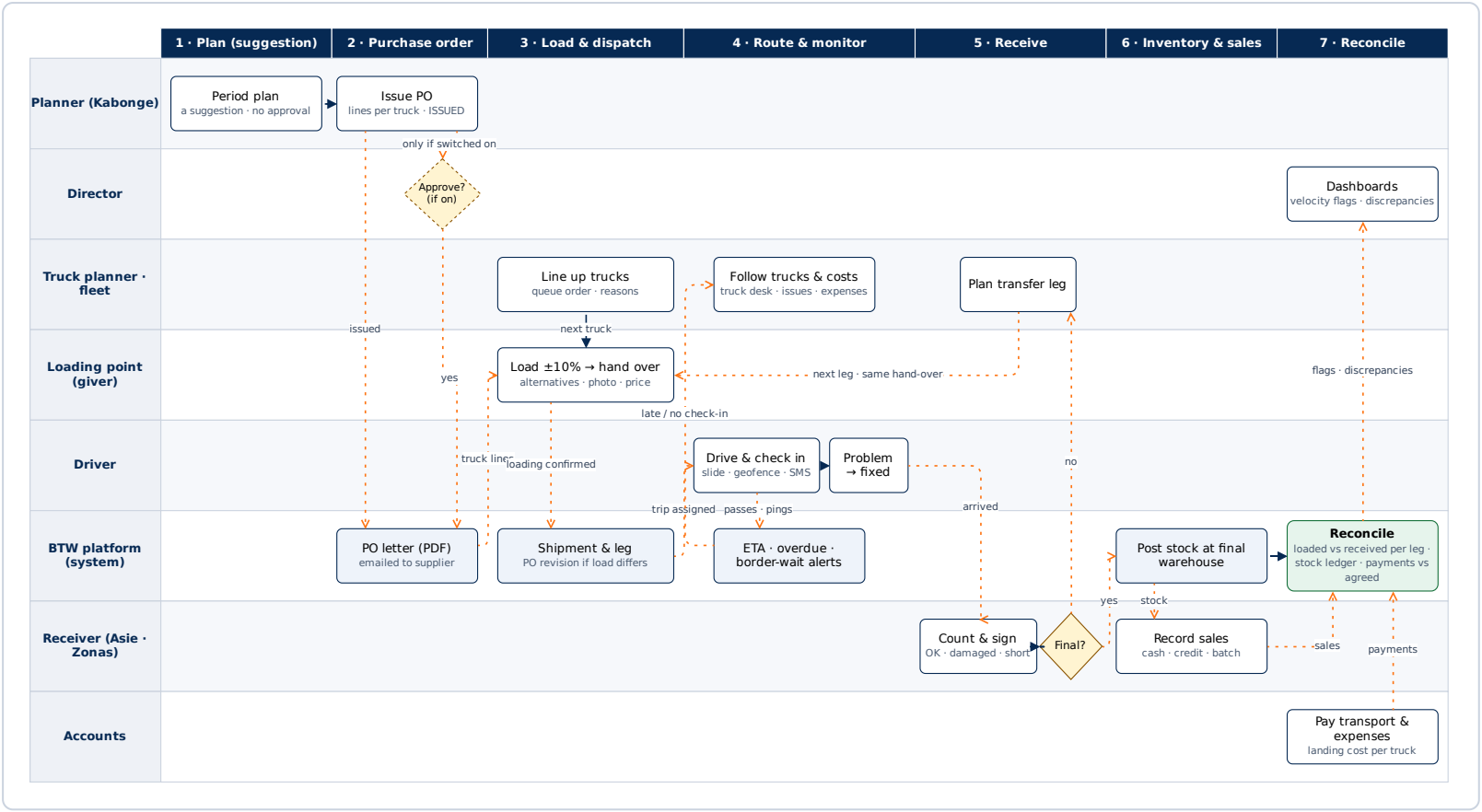
`pnpm start` starts the database and email inbox (Docker), migrates, seeds demo data if empty, builds both apps and serves everything from **http://localhost:3101**: `/` web, `/app/` field, `/brand/` logos, `/v1` API.

Quality gates. `pnpm verify` (type-check, workflow test, API test, builds, size budget) · `test:qa` 638 API, security and integration checks (all passing) · `pnpm e2e` 95 browser steps across 7 user journeys.

2 · PROCESS

From planning to reconciliation, by actor

diagrams/02-process.drawio



Read left to right. Each lane is a person or the system. Solid moving arrows are steps one person does in order; dotted orange moving arrows are hand-overs and notifications between people. Diamonds are decisions: the director's PO approval (dashed: only when `purchase_orders = true`; by default the planner's PO is issued at once) and the receiver's "final destination?" which either posts stock or starts a transfer leg. The truck planner lines up the queue and follows trucks and their costs.

2 · PROCESS

Process steps and reconciliations

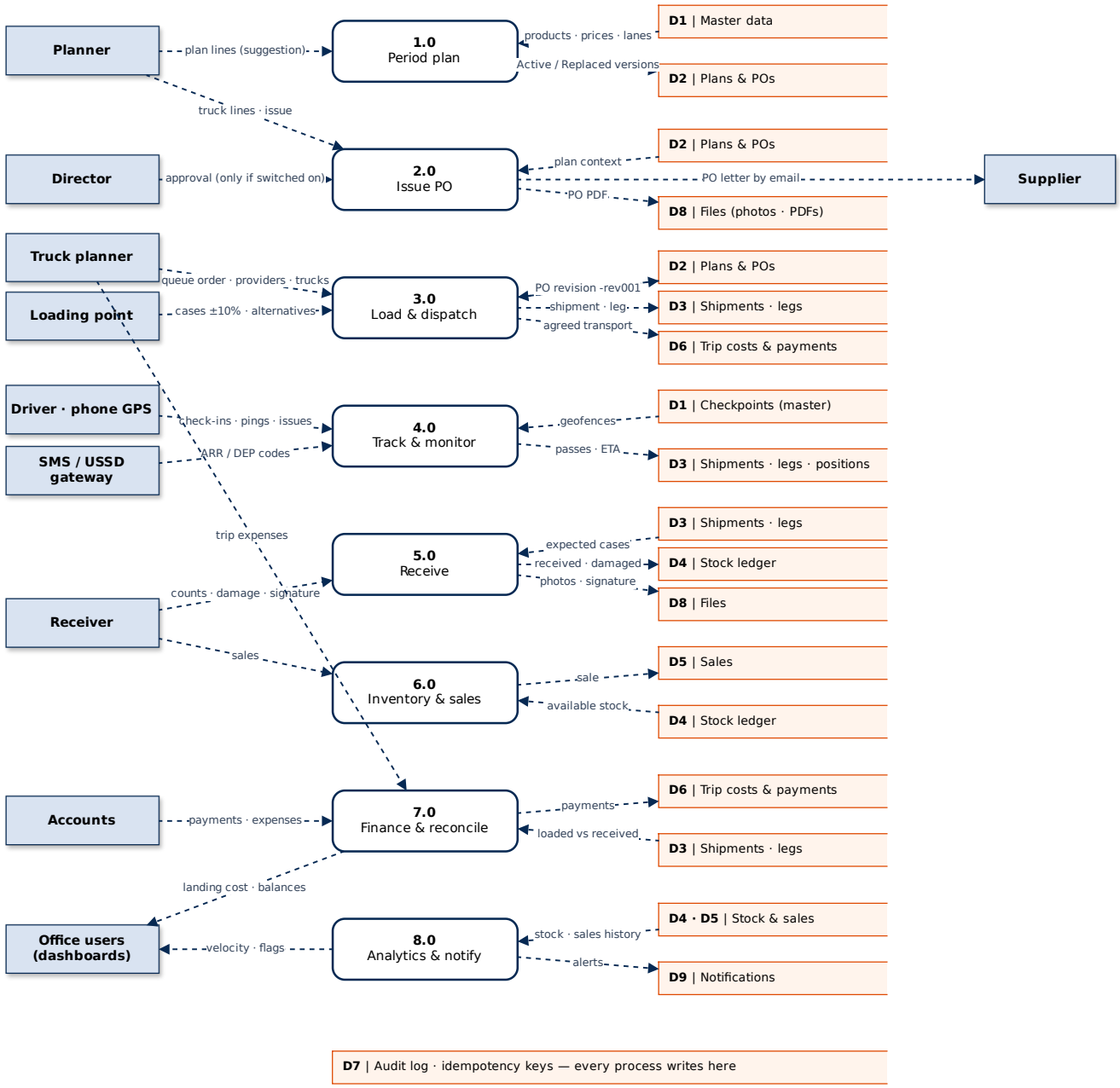
PHASE	WHO	WHAT HAPPENS	SYSTEM RECORD & CONTROL
1 Period plan	Planner	A suggestion per company and period. No approval; each save is a version. It gates nothing.	Active / Replaced; no overlapping Active plans (409 → "Replace?").
2 Purchase order	Planner (→ Director if switched on)	Planner fills truck lines and taps *Issue purchase order*. Letter on the company letterhead, emailed to the supplier.	Ref per company and year (BTW-PO-2026-0002); "ISSUED" stamp, or director approval when `purchase_orders = true`.
3 Load & dispatch	Truck planner · Loading point	Truck planner onboards providers and trucks and orders the queue. The loader loads from the PO within ±10%, may load alternatives, adds photo and agreed price, slides *Hand over*.	Any difference → PO revision `…-rev001` (auto-issued, notified); shipments follow the latest revision.
4 Route & monitor	Driver · Truck planner · Fleet	Driver checks in at each checkpoint (slide, automatic geofence, or SMS/USSD), reports problems and "fixed".	ETA early / on time / late; overdue and border-wait alerts.
5 Receive	Receiver	"Everything arrived OK", or adjust received / damaged per brand with photos; signs.	Discrepancy alert; at a transfer point the next leg is planned.
6 Inventory & sales	System · Receiver	Stock posts only at the final warehouse after acknowledgement; sales reduce stock.	Ledger per warehouse and SKU; sale above stock blocked unless a director overrides.
7 Reconcile	Accounts · Director	Transport payments and expenses recorded; dashboards show velocity flags and discrepancies.	Payments above the threshold need director approval; landing cost per truck.

Reconciliations the platform performs

CHECK	RULE	WHERE IT SHOWS
Ordered vs loaded	Truck total within `±allowance_pct` of the PO and never above capacity; otherwise refused. A difference creates the next revision, numbered from the original PO; superseded orders are not counted twice.	Allowance meter at hand-over, PO revision chain
Loaded vs received, per leg	Each leg: loaded − received = short; damaged counted separately; the next leg carries only good cases forward, so a loss is pinned to the leg where it happened.	Shipment reconciliation table, discrepancy alert
Stock ledger	Opening + received + transfers in − transfers out − damaged ± adjustments − sold = closing; in-transit shown separately.	Inventory (with ledger per SKU)
Transport fees	Agreed price at loading vs payments made; balance and follow-up date per trip.	Trip financials, balances by provider
Landing cost	Goods value + transport + taxes & duties + clearing + loading/offloading + storage = landing cost; ÷ cases = cost per case for stock value.	Trip financials, inventory value
Offline replay	Each phone action has one key; replays apply once and return the stored result.	Sync screen, audit log

Data flow diagram (level 1)

diagrams/03-data-flow.drawio



Boxes (blue) are people and outside systems that send or receive data.

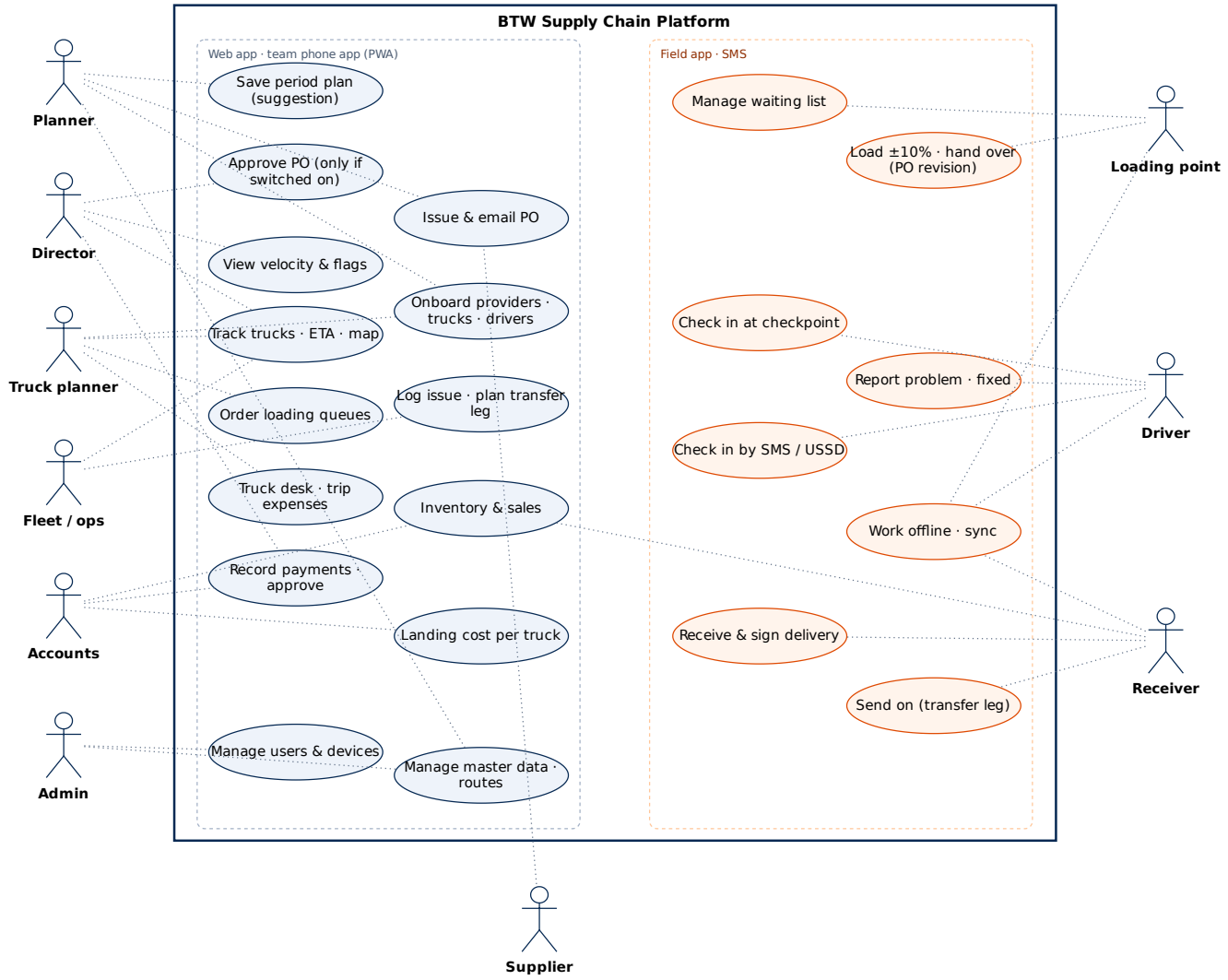
Rounded boxes 1.0–8.0 are the platform's processes; each maps to a service in the API.

Open boxes D1–D9 are data stores; the tables behind each are listed on pages 8–9.

4 · USE CASES

Who does what

diagrams/04-use-cases.drawio



Office roles (left) work in the web app or the team phone app (PWA). **Field roles** (right) use one task-focused screen each in the field app; drivers without data use SMS or USSD. Every action is limited to the person's scope: their entities, loading points or warehouses.

5 · DATA STRUCTURES

Data kept per module

Tables in PostgreSQL (`apps/api/src/db/schema.ts`). *Italic* fields are JSON lists stored inside the record.

Identity & access D1, D7

TABLE	KEY FIELDS
`users`	id, username, name, phone, password hash, PIN hash, roles[], entity_ids[], loading_point_ids[], warehouse_ids[], driver_id, active, token_version
`devices` · `refresh_tokens`	device id, user, push token, platform, app version, last seen, revoked at · token hash, device, expiry, revoked at

Master data D1

TABLE	KEY FIELDS
`entities`	code (BTW, NUD, KVD), name, address, PO prefix
`receiving_companies` · `warehouses`	Asie, Zonas, countries · warehouse name, receiving company, checkpoint
`suppliers` · `products` · `price_list`	code (KBL, UBL, SBL, GBI), city, email, checkpoint · SKU, brand, pack (24 × 330 ml), units per case · price, currency, effective from
`providers` · `trucks` · `drivers`	name, kind (company / sacco / individual / market), contact, email, notes, regular, active · registration, trailer, capacity in cases, default driver, documents, active · name, phone, licence, has app, active
`checkpoints` · `routes`	name, kind (loading, weighbridge, border, warehouse), code (MALABA), lat/lng, geofence radius, geography · lane code, distance, *stops: checkpoint, planned hours, planned dwell*

Planning & purchase orders D2

TABLE	KEY FIELDS
`period_plans` · `period_plan_versions`	ref, entity, period kind, from / to, status (Active / Replaced), replaced by, version, *lines*, *history* · snapshot of every save
`plans` · `plan_versions`	PO order: ref, entity, supplier, status (Draft → Submitted → Approved / Issued), version, *lines: truck no, product, cases, price, to, route*, *history*, revision of / no., superseded by · snapshots
`pos`	ref, plan, entity, supplier, approved / issued by and at, signature, approval (director / issued / revision), revision of / no., *changes: truck, product, ordered → loaded, kind*, reason, on behalf of, superseded by

Loading & logistics D3

TABLE	KEY FIELDS
`waiting_entries`	loading point, truck, driver, arrived at, position, status (Waiting / Assigned / Loaded)
`loadings`	PO, truck no, truck, driver, loading point, loaded at, *lines: ordered vs loaded cases*, agreed transport + currency, *documents*, status
`shipments`	ref (SHP-2026-0041), PO, truck no, entity, loading point, receiving company, final warehouse, status (derived from legs)
`legs`	shipment, seq, route, from / to, truck, driver, status, departed, planned arrival, *passes: checkpoint, planned, arrived, departed, source*, last ping, *lines*, *issues: place, reason, fixed at, new due date*, acknowledgement
`positions`	leg, truck, lat/lng, geography, time, source (app / GPS provider), accuracy, speed

Data kept per module (continued)

Receipt & inventory D4, D8

TABLE	KEY FIELDS
`acks`	leg, shipment, warehouse, receiver name, time, *lines: product, expected, received, damaged, remarks*, signature, *photos*, has discrepancy, by user
`stock_moves`	warehouse, product, kind (received, damaged, sold, transfer in / out, adjustment), cases, time, reference — the ledger behind every stock figure

Sales D5

TABLE	KEY FIELDS
`sales`	ref, time, warehouse, customer, salesperson, kind (cash / credit / batch), *lines: product, cases, price*, currency, paid, status (Paid / Part-paid / Unpaid)

Finance & landing cost D6

TABLE	KEY FIELDS
`trip_costs`	shipment, leg, kind (transport, taxes & duties, clearing & forwarding, loading & offloading, storage, other), amount, currency, document ref
`trip_payments`	shipment, cost line, amount, time, reference (M-Pesa / bank), status (Pending approval / Approved / Paid), approved by, receipt file

Platform: sync, audit, notifications, files D7, D8, D9

TABLE	KEY FIELDS
`idempotency_keys`	key, user, operation, status (applied / duplicate / rejected), stored response — makes offline replays safe
`audit_log`	time, user, channel (web / app / sms), method, path, status, detail
`notifications` · `notification_reads`	title, body, link, for roles or for one user · who has read it
`files` · `settings`	name, type, size, url, uploaded by (photos, signatures, PDFs) · thresholds and other director settings

Not built yet

A separate reporting store for heavy dashboards (they compute live today), paged lists, SMS delivery of new PINs, and the management accounts per receiving company (P&L, retained earnings). Real SMS and push keys still need to be tested.